KloudOcean
THEKLOUDOCEAN.COM

Teradata → BigQuery SQL Cheat Sheet

25 real syntax differences — what breaks, what ports cleanly, and the failure mode each one causes.

1. Current date

```
TD   CURRENT_DATE
BQ   CURRENT_DATE()
```

BigQuery requires the parentheses; bare CURRENT_DATE fails to parse in some contexts.

2. Current timestamp

```
TD   CURRENT_TIMESTAMP
BQ   CURRENT_TIMESTAMP()
```

Same rule as CURRENT_DATE — always call it as a function in BigQuery.

3. Add days to a date

```
TD   date_col + 30
BQ   DATE_ADD(date_col, INTERVAL 30 DAY)
```

BigQuery has no implicit integer-to-date arithmetic; every offset needs an explicit interval.

4. Difference between dates

```
TD   date2 - date1
BQ   DATE_DIFF(date2, date1, DAY)
```

Direct subtraction on DATE types isn't valid in BigQuery Standard SQL.

5. Zero-safe division

```
TD   ZEROIFNULL(col)
BQ   IFNULL(col, 0)
```

No direct ZEROIFNULL — rewrite explicitly per column, and check every usage; the intent is easy to lose in translation.

6. Null-out a sentinel value

```
TD   NULLIFZERO(col)
BQ   NULLIF(col, 0)
```

Same family of TD shorthand functions — none of them exist natively in BigQuery.

7. Top-N rows

```
TD   SELECT TOP 10 * FROM t
BQ   SELECT * FROM t LIMIT 10
```

TOP is not valid BigQuery syntax; also check for ORDER BY — TOP without one is non-deterministic in both systems.

8. Row sampling

```
TD   SAMPLE 100
BQ   TABLESAMPLE SYSTEM (10 PERCENT)
```

Different semantics: TD SAMPLE returns an exact row count; BigQuery TABLESAMPLE is block-level and percentage-based, not exact.

Teradata → BigQuery SQL Cheat Sheet

(continued)

9. Set difference

```
TD   SELECT ... MINUS SELECT ...
BQ   SELECT ... EXCEPT DISTINCT SELECT ...
```

MINUS is not a BigQuery keyword — EXCEPT DISTINCT is the equivalent.

10. Set intersection

```
TD   SELECT ... INTERSECT SELECT ...
BQ   SELECT ... INTERSECT DISTINCT SELECT ...
```

BigQuery requires an explicit DISTINCT or ALL qualifier; bare INTERSECT will not parse.

11. Fixed-width string cast

```
TD   col (FORMAT 'X(10)')
BQ   LPAD(col, 10) / RPAD(col, 10)
```

TD's inline FORMAT clause has no BigQuery equivalent — pad explicitly with LPAD/RPAD.

12. Parse a formatted date string

```
TD   CAST(col AS DATE FORMAT 'YYYY-MM-DD')
BQ   PARSE_DATE('%Y-%m-%d', col)
```

BigQuery uses strftime-style format tokens, not TD's FORMAT strings — every FORMAT clause needs manual translation.

13. Currency-style number format

```
TD   CAST(amt AS FORMAT '$$,$$9.99')
BQ   FORMAT('%\'d', amt) or app-layer formatting
```

BigQuery has no built-in currency mask; most teams move this formatting to the BI layer instead.

14. Auto-incrementing key

```
TD   GENERATED ALWAYS AS IDENTITY
BQ   GENERATE_UUID() or ROW_NUMBER()-based surrogate
```

No native auto-increment in BigQuery — pick a surrogate key strategy deliberately, don't default to UUID without checking join performance impact.

15. Volatile / temp tables

```
TD   CREATE VOLATILE TABLE ... ON COMMIT PRESERVE ROWS
BQ   CREATE TEMP TABLE (session-scoped)
```

Semantics differ: TD volatile tables are transaction-and-session scoped with COMMIT-driven row retention; BigQuery temp tables are script/session scoped with no COMMIT PRESERVE/DELETE distinction.

16. Row-level security

```
TD   View or macro with a WHERE-clause filter
BQ   CREATE ROW ACCESS POLICY ...
```

Move implicit, view-based masking to BigQuery's native row access policies — don't just recreate the view and call it done.

Teradata → BigQuery SQL Cheat Sheet

(continued)

17. Distribution vs. clustering

```
TD   PRIMARY INDEX (col)
BQ   CLUSTER BY col
```

Different mechanics: TD's PI drives physical row distribution and join colocation; BigQuery clustering sorts storage blocks for pruning and does not guarantee join colocation the same way. A common source of join performance regressions post-migration.

18. Query plan inspection

```
TD   EXPLAIN SELECT ...
BQ   Execution Details tab / INFORMATION_SCHEMA.JOBS
```

EXPLAIN isn't a BigQuery keyword in the same sense — use the console's Execution Graph or query the JOBS metadata table.

19. Pivoting rows to columns

```
TD   Manual CASE-based pivot logic
BQ   PIVOT(...)
```

BigQuery's native PIVOT operator usually replaces pages of hand-written CASE statements common in older BTEQ scripts — worth refactoring, not just porting.

20. Merge / upsert

```
TD   MERGE INTO t USING s ON ... WHEN MATCHED ...
BQ   MERGE t USING s ON ... WHEN MATCHED ...
```

Close to a direct port — standard SQL MERGE syntax is similar in both. Still verify matched/not-matched clause ordering and column list alignment.

21. Multi-statement transactions

```
TD   BT; stmt1; stmt2; ET;
BQ   Scripted multi-statement query or stored procedure
```

BigQuery has no session-level BT/ET transaction block — atomicity across multiple statements needs a script, a stored procedure, or a different design entirely.

22. Recursive queries

```
TD   WITH RECURSIVE cte AS (...)
BQ   WITH RECURSIVE cte AS (...)
```

Genuinely compatible — BigQuery supports recursive CTEs with near-identical syntax. One of the few patterns that ports with minimal change.

23. CASE expressions

```
TD   CASE WHEN ... THEN ... END
BQ   CASE WHEN ... THEN ... END
```

Transfers directly. Don't waste conversion effort rewriting what already works.

24. Coalesce / null defaults

```
TD   COALESCE(col1, col2, 0)
BQ   COALESCE(col1, col2, 0)
```

Also transfers directly — standard ANSI function, no change needed.

Teradata → BigQuery SQL Cheat Sheet

(continued)

25. String concatenation

```
TD   col1 || col2
BQ   CONCAT(col1, col2) or col1 || col2
```

Both operators work in BigQuery. Prefer CONCAT for clarity when mixing types, since implicit casting rules differ from Teradata's.